

Projeto Django — Lista de Tarefas

Projeto simplificado para uma aula de aproximadamente **1 hora**, utilizando Django, SQLite, HTML e CSS.

1. Criar a pasta do projeto

Criar a pasta `taskflow` e entrar nela.

Windows — CMD/PowerShell:

Bash

```
mkdir taskflow  
cd taskflow
```

Bash:

Bash

```
mkdir taskflow  
cd taskflow
```

2. Criar o ambiente virtual

Windows:

Bash

```
python -m venv .venv
```

Bash:

Bash

```
python3 -m venv .venv
```

3. Ativar o ambiente virtual

Windows — CMD:

Bash

```
.env\Scripts\activate
```

Windows — PowerShell:

PowerShell

```
.env\Scripts\Activate.ps1
```

Bash:

Bash

```
source .env/bin/activate
```

4. Instalar o Django

Windows:

Bash

```
python -m pip install django
```

Bash:

Bash

```
python3 -m pip install django
```

Verificar a instalação:

Bash

```
python -m django --version
```

5. Criar o projeto Django

Bash

```
django-admin startproject config .
```

Criar a aplicação:

Bash

```
python manage.py startapp tasks
```

No Bash, caso necessário:

Bash

```
python3 manage.py startapp tasks
```

6. Configurar o aplicativo

Abrir:

```
config/settings.py
```

Adicionar `tasks` em `INSTALLED_APPS`.

Arquivo: `settings.py`

Faça o download do arquivo `settings.py` e substitua o arquivo existente.

O `settings.py` também deve conter a configuração para funcionamento no GitHub Codespaces:

```
ALLOWED_HOSTS
CSRF_TRUSTED_ORIGINS
templates
```

7. Criar o Model

Abrir:

```
tasks/models.py
```

Criar o modelo `Task`, contendo:

- `title` — título da tarefa;
- `completed` — indica se a tarefa foi concluída.

Faça o download do arquivo `models.py` e substitua o arquivo existente.

8. Criar o banco de dados

Executar:

Bash

```
python manage.py makemigrations  
python manage.py migrate
```

Isso criará o banco de dados SQLite utilizado pelo projeto.

9. Criar as Views

Abrir:

```
tasks/views.py
```

Criar as funcionalidades para:

- listar tarefas;
- criar tarefas;
- concluir/desmarcar tarefas;
- excluir tarefas.

Faça o download do arquivo `views.py` e substitua o arquivo existente.

10. Criar as URLs da aplicação

Criar:

```
tasks/urls.py
```

Esse arquivo será responsável por direcionar as requisições para as respectivas Views.

Faça o download do arquivo `urls.py` e coloque-o dentro da pasta `tasks`.

11. Configurar as URLs principais

Abrir:

```
config/urls.py
```

Configurar o projeto para utilizar as URLs da aplicação `tasks`.

Faça o download do arquivo `urls.py` e substitua o arquivo existente dentro da pasta `config`.

12. Criar o HTML

Criar a seguinte estrutura:

```
templates/  
└─ tasks/  
    └─ task_list.html
```

O arquivo HTML será responsável pela interface da aplicação.

Ele permitirá:

- inserir uma nova tarefa;
- visualizar as tarefas;
- concluir uma tarefa;
- excluir uma tarefa.

Faça o download do arquivo `task_list.html` e coloque-o em `templates/tasks/`.

13. Configurar os Templates

No:

```
config/settings.py
```

Verificar se o Django está configurado para procurar os templates na pasta:

```
templates/
```

Essa configuração já estará presente no `settings.py` fornecido no **passo 6**.

14. Executar o projeto

Iniciar o servidor:

Windows:

Bash

```
python manage.py runserver 0.0.0.0:8000
```

Bash:

Bash

```
python3 manage.py runserver 0.0.0.0:8000
```

No GitHub Codespaces, abrir a porta **8000** através da aba **PORTS**.

15. Testar a aplicação

Testar as seguintes funcionalidades:

- ☐ Abrir a página inicial.
- ☐ Criar uma tarefa.
- ☐ Verificar se a tarefa aparece na lista.
- ☐ Marcar a tarefa como concluída.
- ☐ Desmarcar a tarefa.
- ☐ Excluir a tarefa.
- ☐ Criar mais de uma tarefa.

A aplicação deverá apresentar uma interface semelhante a:

Lista de Tarefas

[Digite uma tarefa] [Adicionar]

Estudar Django Concluir | Excluir

Fazer exercício Concluir | Excluir

Ler documentação Concluir | Excluir

16. Estrutura final do projeto

Ao final, a estrutura deverá ser:

```
taskflow/
|
├─ .venv/
|
├─ config/
|   ├── __init__.py
|   ├── settings.py
|   ├── urls.py
|   ├── asgi.py
|   └── wsgi.py
|
├─ tasks/
|   ├── migrations/
|   │   └── __init__.py
|   ├── __init__.py
|   ├── admin.py
|   ├── apps.py
|   ├── models.py
|   ├── tests.py
|   ├── urls.py
|   └── views.py
|
├─ templates/
|   └── tasks/
|       └── task_list.html
|
├─ db.sqlite3
└─ manage.py
```

Fluxo básico do Django

```
Usuário
  ↓
URL
  ↓
VIEW
  ↓
MODEL
  ↓
Banco de dados
  ↓
VIEW
  ↓
TEMPLATE
  ↓
Usuário
```

Objetivo da aula: ao final, o aluno deve compreender como as principais partes de uma aplicação Django se conectam para criar uma aplicação web funcional.